

WHERE TO GO

AI 교육 컨설턴트 서비스

통합 기획 및 설계서

기획 · 요구사항 · UX · AI/RAG · 데이터 · ERD · API · 학습관리 · 협업

프로젝트명	WhereToGo - 교육 컨설턴트 AI
프로젝트 기간	2026.05.08 ~ 2026.06.25
문서 기준일	2026.06.24
기술 스택	Django · Vue · OpenAI · LangGraph · ChromaDB
문서 상태	실제 구현 검증 반영 최종 보완본
검증 기준 코드	C:\Users\SSAFY\Desktop\WhereToGo_PJT

Django REST FrameworkVue 3RAGAgentStreamingHttpResponseLearning Dashboard

문서 개요

본 문서는 WhereToGo 프로젝트의 기획, 요구사항, 시스템 설계, AI/RAG 설계, 데이터베이스 설계, API 계약, 프론트엔드 화면 구성, 학습 실행 설계, 테스트 및 협업 방식을 하나로 정리한 제출용 통합 문서이다.

원본 설계서의 방향성을 유지하되, 최종 코드와 대조하여 실제 구현과 다른 초기 기획 항목은 "향후 고도화"로 이동하고, 실제 구현된 학습 대시보드 API, 스트리밍 생성 UX, 데모 데이터 seed 등 누락된 구현 사항을 반영했다.

작성 원칙

- 실제 코드에 존재하는 모델, API, 화면, 서비스 계층을 우선 기준으로 삼는다.
- 구현되지 않은 면담 세션 DB, 진단 퀴즈, 학습 노트 전용 모델은 핵심 구현에서 제외하고 향후 확장으로 둔다.
- AI가 만든 자연어 결과를 그대로 저장하지 않고, 검색 결과 식별자와 FK 변환을 통해 데이터 무결성을 확보한 구조를 설명한다.
- 배포 인프라는 현재 확정 구현이 없으므로 로컬 실행 기준과 미배포 상태를 명시한다.

목차

1장	프로젝트 개요와 문제 정의	10장	커리큘럼 생성·저장 설계
2장	목표·범위·성공 기준	11장	학습 시작·일시정지·완료 설계
3장	사용자와 서비스 시나리오	12장	API 설계
4장	기능 요구사항과 MVP	13장	프론트엔드 설계
5장	UX/UI 및 정보 구조	14장	비기능 요구사항·오류 처리
6장	시스템 아키텍처	15장	테스트 및 검증 계획
7장	AI·RAG·Agent 설계	16장	협업·Git·WBS
8장	데이터셋 및 Topic Catalog	17장	설계 변경 이력
9장	데이터베이스·ERD 설계	18장	한계와 향후 고도화

1. 프로젝트 개요와 문제 정의

1.1 서비스 한 줄 정의

WhereToGo는 사용자의 학습 목표와 현재 조건을 바탕으로 실제 교육 데이터에 근거한 커리큘럼과 학습 자료를 추천하고, 학습 시작 이후 진행 상태까지 관리하는 개인 학습자용 AI 교육 컨설턴트 서비스이다.

1.2 문제 배경

학습자는 새로운 기술이나 분야를 배우려 할 때 무엇부터 시작해야 하는지, 어떤 자료가 적합한지, 어느 순서로 학습해야 하는지를 먼저 결정해야 한다. 이 과정은 학습 그 자체보다 많은 탐색과 판단을 요구하며, 특히 잘 모르는 분야일수록 계획 수립 단계에서 이탈하기 쉽다.

범용 생성형 AI를 사용하더라도 같은 목표에 대해 결과가 크게 달라질 수 있고, 실제로 존재하지 않는 자료를 추천하거나 학습 순서와 난이도가 일관되지 않는 문제가 발생한다. 따라서 본 프로젝트는 "학습 계획 수립의 인지 비용"과 "AI 추천의 신뢰성"을 동시에 해결하는 것을 목표로 한다.

1.3 핵심 문제 정의

문제	발생 결과	설계 대응
시작점 불명확	학습 목표는 있으나 첫 행동으로 이어지지 않음	질문 흐름과 단계별 로드맵 자동 생성
탐색 비용 과다	강의와 문서를 비교하는 데 많은 시간 소요	교육 데이터 검색과 추천 자료 연결 자동화
수준·시간 미반영	과도하거나 부족한 학습량	수준, 기간, 주간 가능 시간을 입력으로 반영
순수 LLM 불안정성	할루시네이션과 결과 일관성 저하	Topic Catalog, RAG 검색, 식별자 검증 결합
계획과 실행의 단절	생성된 커리큘럼이 실제 학습 관리로 이어지지 않음	StepProgress, Schedule, Progress 모델 분리

2. 목표·범위·성공 기준

2.1 목표 서비스

- 사용자의 학습 목표, 목적, 수준, 기간, 주간 가능 시간, 선호 학습 방식을 입력받는다.
- 보유한 강의계획서와 공개 강좌 데이터를 RAG로 검색한다.
- 검색 결과를 근거로 단계별 커리큘럼, 추천 강의, 추천 학습 자료를 생성한다.
- 사용자가 preview를 확인한 뒤 저장하면 실제 DB row로 반영한다.
- 저장된 커리큘럼을 시작, 일시정지, 재개, 완료할 수 있다.
- 학습 대시보드에서 현재 학습, 진행률, 최근 활동, 로드맵을 확인한다.

2.2 구현 범위

범위	구현 여부	설명
회원가입/로그인	완료	Simple JWT 기반 인증
사용자 프로필	완료	주간 학습 가능 시간, 선호 학습 방식 저장
프론트 질문 흐름	완료	<code>/chat</code> , <code>/interview/summary</code> 화면 기반
백엔드 면담 세션 저장	미구현	향후 <code>interview_sessions</code> 확장 가능
RAG + Agent 생성	완료	LangGraph, ChromaDB, OpenAI 사용
Streaming 생성 UX	완료	<code>StreamingHttpResponse</code> + 프론트 <code>ReadableStream</code>
커리큘럼 저장/상세	완료	Preview와 Save 분리
학습 시작/정지/재개/완료	완료	현재 Step 중심 일정 생성
학습 대시보드	완료	요약, 현재 학습, 진행률 목록, 로드맵 API
배포	미배포	현재 로컬 실행 기준

2.3 성공 기준

- 동일한 사용자 입력에 대해 실제 존재하는 강의/자료 식별자를 기반으로 커리큘럼이 생성된다.
- 생성 결과에 검색 결과에 없는 `course/resource identifier`가 포함되면 제거되거나 저장되지 않는다.
- 사용자는 생성 중 progress bar와 문구를 통해 서비스가 동작 중임을 인지할 수 있다.
- 학습 시작 시 전체 미래 일정을 만들지 않고 현재 Step 일정만 생성한다.
- 대시보드에서 진행 중, 완료, 일시정지, 초안 커리큘럼 상태가 구분된다.

3. 사용자와 서비스 시나리오

3.1 핵심 사용자

사용자	상황	필요
취업 준비 학습자	포트폴리오를 만들고 싶지만 학습 순서가 불명확함	실습 중심 커리큘럼과 추천 강의
전환 학습자	비전공 또는 다른 분야에서 데이터/AI로 이동하려 함	초급 개념부터 단계별 자료
반복 학습자	여러 커리큘럼을 병행하거나 중단 후 재개함	현재 학습과 진행률 대시보드

3.2 주요 시나리오

- 사용자가 홈 화면에서 학습 목표를 입력한다.
- `/chat` 화면에서 목적, 수준, 기간, 주간 가능 시간, 선호 학습 방식을 선택한다.
- `/interview/summary` 에서 답변을 확인하고 커리큘럼 생성을 요청한다.
- 백엔드는 streaming API로 진행 상태를 전달하고 프론트는 progress bar와 문구를 갱신한다.
- 생성 결과는 `/curriculum/result` 에서 preview로 표시된다.
- 사용자가 저장하면 `/api/curriculums/save-generated/` 가 호출되어 DB에 반영된다.
- 사용자는 `/curriculum/:id` 에서 학습을 시작하고 Step 완료를 기록한다.
- `/learning` 대시보드에서 현재 학습, 최근 활동, 진행률을 확인한다.

4. 기능 요구사항과 MVP

기능	MVP 요구사항	실제 구현
인증	회원가입, 로그인, 토큰 갱신	<code>/api/auth/signup/</code> , <code>/api/auth/login/</code> , <code>/api/auth/token/refresh/</code>
프로필	학습 가능 시간과 선호 방식 관리	<code>/api/users/me/profile/</code>
Topic	학습 개념 표준화	<code>Topic</code> , <code>TopicAlias</code> , <code>/api/topics/</code>
AI 생성	입력 조건 기반 커리큘럼 생성	<code>generate_curriculum</code> , <code>generate_curriculum_stream</code>
저장	AI preview를 사용자가 확정 후 저장	<code>save_ai_generated_curriculum</code>
학습 실행	현재 Step 시작/정지/완료	<code>curriculum_learning_service.py</code>
대시보드	현재 학습과 진행 현황 확인	<code>learning_dashboard_service.py</code>

5. UX/UI 및 정보 구조

5.1 실제 화면 구조

경로	화면	역할
<code>/</code>	HomePage	목표 입력과 서비스 진입
<code>/signup</code> , <code>/login</code>	Auth pages	인증
<code>/mypage</code>	MyPage	프로필과 기본 학습 조건
<code>/chat</code>	ChatInterview	질문 기반 조건 수집
<code>/interview/summary</code>	InterviewSummary	답변 확인, 생성 요청, streaming 대기 화면
<code>/curriculum/result</code>	CurriculumResult	생성 결과 preview와 저장
<code>/curriculum/:id</code>	CurriculumDetailPage	상세, 로드맵, 학습 제어
<code>/learning</code>	LearningDashboard	개인 학습 대시보드

5.2 UX 원칙

- 생성 중에는 별도 full-screen 대기 화면과 progress bar로 사용자 불안을 줄인다.
- 커리큘럼 상세 화면은 요약, 진행률, Step, 추천 자료, 학습 CTA를 한 흐름으로 제공한다.
- 사이드바는 커리큘럼 목록과 진행률을 표시하고, 페이지별 스타일 편차를 줄이도록 공통 shell 스타일을 사용한다.
- 학습 대시보드는 현재 이어서 할 학습과 전체 진행 상황을 먼저 보여준다.

6. 시스템 아키텍처

계층	구성	책임
Frontend	Vue, Vue Router, Axios/fetch	질문 흐름, 생성 상태, 결과, 학습 대시보드 UI
Backend API	Django, DRF	권한 검증, serializer, service 호출, 응답 직렬화
Domain Service	apps/curriculum/services	커리큘럼 저장, 학습 상태 변경, 대시보드 집계
AI Orchestration	LangGraph	입력 분석, 검색, 생성 단계 조율
RAG	ChromaDB, OpenAI embeddings	강의계획서와 학습 자료 의미 검색
Database	SQLite	사용자, Topic, 커리큘럼, 일정, 진행 기록 저장
Streaming	StreamingHttpResponse	장시간 생성 과정의 progress/done/error 전달

6.1 실제 프로젝트 구조

```
WhereToGo_PJT/  
├─ manage.py  
├─ config/  
├─ apps/  
│   ├── accounts/  
│   └── curriculum/  
├─ ai/  
│   ├── agent/  
│   ├── core/  
│   ├── indexing/  
│   └── search/  
├─ scripts/topic_pipeline/  
├─ curriculum-data/  
└─ frontend/
```

7. AI·RAG·Agent 설계

7.1 RAG + Agent 선택 이유

대안	장점	한계	판단
Rule-based	결과가 명확함	새 목표와 맥락 확장에 약함	보조 검증
Keyword search	빠르고 단순함	동의어, 순서, 난이도 반영 부족	Topic 매칭 보조
Pure LLM	자연어 생성 품질 높음	할루시네이션, 실제 자료 존재 여부 문제	단독 사용하지 않음
RAG + Agent	근거 검색과 단계별 판단 가능	데이터 품질과 검증 로직 필요	최종 채택

7.2 실제 구현 흐름

사용자 입력

```
-> CurriculumGenerateSerializer.to_raw_input()
-> build_topic_catalog()
-> ai.agent.run_agent(raw_input, catalog)
-> input_analysis node
-> search node
-> curriculum generation node
-> identifier validation
-> normalize_agent_response()
-> preview 반환 또는 save-generated 저장
```

7.3 RAG 설정

- Vector DB: ChromaDB
- Collection: `courses`, `resources`
- Embedding: `text-embedding-3-small`
- Chat model: `gpt-4o-mini`
- 문서 단위: CSV row 하나를 하나의 검색 문서로 사용

8. 데이터셋 및 Topic Catalog

데이터	역할	실제 위치/모델
LearningResource	KOCW/K-MOOC 등 온라인 학습 자료	<code>learning_resources</code> , <code>ai/data/learning_resource.csv</code>
CurriculumCourse	대학 강의계획서 기반 과목 데이터	<code>curriculum_courses</code> , <code>ai/data/curriculum_courses.csv</code>
Topic	학습 개념 표준화	<code>topics</code>
TopicAlias	한글/영문/약어/변형 표현 통합	<code>topic_aliases</code>
CourseTopic/ResourceTopic	강의/자료와 Topic 연결	<code>course_topics</code> , <code>resource_topics</code>

8.1 Topic 구축 정책

- MVP 기준 최대 depth는 3으로 제한한다.
- 표준명은 한국어 중심으로 두고, 영어/약어/변형 표현은 alias로 관리한다.
- 새 후보를 즉시 `topics`에 넣지 않고, 후보 검토 pipeline을 거친다.
- LLM은 topic 매칭과 후보 추천을 보조하되 자동 승인하지 않는다.

9. 데이터베이스·ERD 설계

9.1 실제 구현 테이블

도메인	테이블	설명
사용자	users , user_profiles , user_interest_topics	계정, 학습 선호, 관심 topic
Topic	topics , topic_aliases	학습 개념과 별칭
교육 데이터	learning_resources , curriculum_courses	추천 자료와 강의계획서
Topic 연결	resource_topics , course_topics	자료/강의와 Topic의 다대다 연결
커리큘럼	curricula , curriculum_categories , curriculum_steps	사용자별 계획과 단계
추천 근거	curriculum_step_resources , curriculum_step_courses	Step별 추천 자료/강의
학습 실행	curriculum_step_progresses , learning_schedules , learning_progresses	진행 상태, 일정, 실제 기록

9.2 현재 구현에서 제외된 확장 엔티티

원본 설계서에 포함되었던 interview_sessions , interview_messages , diagnostic_questions , diagnostic_attempts , diagnostic_answers , learning_notes , user_skill_profiles 는 현재 Django 모델로 구현되어 있지 않다. 최종 ERD에서는 핵심 구현 테이블에서 제외하고 향후 고도화 항목으로 둔다.

9.3 계획·상태·일정·기록 분리

테이블	의미	저장하지 않는 것
curriculum_steps	AI가 생성한 변하지 않는 단계별 계획	사용자의 시작/완료 상태
curriculum_step_progresses	사용자별 Step 실행 상태와 누적 진행	원본 계획 내용
learning_schedules	현재 Step의 날짜/회차별 예정	전체 미래 Step 일정
learning_progresses	실제 학습 시간과 완료 기록	계획 시간 자체

10. 커리큘럼 생성·저장 설계

10.1 생성 입력

프론트는 목표, 목적, 난이도, 기간, 주간 가능 시간, 선호 학습 방식을 전송한다. 백엔드는 CurriculumGenerateSerializer 로 검증한 뒤 Agent 입력인 raw_input 으로 변환한다.

10.2 Preview와 Save 분리

AI 생성 결과는 곧바로 DB에 저장하지 않는다. 생성 API는 사용자가 확인할 preview payload를 반환하고, 사용자가 저장을 선택하면 /api/curriculums/save-generated/ 에서 transaction으로 저장한다.

- 장점: 잘못된 AI 결과가 사용자 DB에 바로 남지 않는다.
- 장점: 저장 전 화면에서 강의/자료 preview를 확인할 수 있다.
- 장점: FK 조회와 transaction을 저장 service에 집중시킬 수 있다.

10.3 StreamingHttpResponse 적용

커리큘럼 생성은 RAG 검색과 LLM 호출 때문에 시간이 걸릴 수 있다. 백엔드는 StreamingHttpResponse 로 progress , done , error 이 이벤트를 전송하고, 프론트는 fetch 와 ReadableStream 으로 chunk를 읽어 progress bar와 문구를 갱신한다.

단계	progress	설명
입력 검증	10	요청 body 검증
Topic catalog 구성	25	AI Agent에 전달할 topic 후보 구성
Agent 실행	45	RAG 검색과 LLM 생성
결과 정규화	80	응답 payload 구성
완료	100	최종 결과 반환

실제 구현에서는 별도 generation_id 상태 조회 API가 없다. 따라서 원본 설계서의 generation 상태 조회는 향후 고도화 항목으로 분리한다.

11. 학습 시작·일시정지·완료 설계

11.1 초기 설계와 문제

초기에는 커리큘럼 시작 시 모든 Step의 미래 일정을 한 번에 생성하려 했다. 그러나 사용자가 중간에 일시정지하거나 특정 Step에 예상보다 오래 걸릴 경우, 이후 모든 일정이 무효화되고 일정 테이블이 상태와 진행률까지 떠안은 문제가 생겼다.

11.2 최종 의사결정

전체 계획은 curriculum_steps 에 먼저 저장하되, learning_schedules 는 사용자가 실제로 시작한 현재 Step에 대해서만 생성한다. Step 실행 상태는 CurriculumStepProgress , 실제 기록은 LearningProgress 가 관리한다.

11.3 상태 흐름

대상	상태 흐름
Curriculum	DRAFT -> ACTIVE -> PAUSED -> ACTIVE -> COMPLETED
StepProgress	NOT_STARTED -> IN_PROGRESS -> PAUSED -> IN_PROGRESS -> COMPLETED
LearningSchedule	PLANNED -> DONE 또는 PLANNED -> CANCELLED
LearningProgress	IN_PROGRESS , PAUSED , PARTIAL , COMPLETED

12. API 설계

12.1 인증·프로필

Method	Endpoint	설명
POST	/api/auth/signup/	회원가입
POST	/api/auth/login/	로그인
POST	/api/auth/token/refresh/	토큰 갱신
GET/PATCH	/api/users/me/	현재 사용자 조회/수정
GET/PATCH	/api/users/me/profile/	학습 프로필 조회/수정
POST	/api/users/me/password/	비밀번호 변경

12.2 Topic

Method	Endpoint	설명
GET	/api/topics/	활성 topic 목록 조회

12.3 커리큘럼·학습

Method	Endpoint	설명
GET/POST	/api/curriculums/	목록 조회 / 일반 생성
GET	/api/curriculums/{id}/	상세 조회
POST	/api/curriculums/generate/	AI 커리큘럼 생성
POST	/api/curriculums/generate/stream/	AI 생성 streaming
POST	/api/curriculums/save-generated/	AI preview 저장
POST	/api/curriculums/{id}/start/	학습 시작
POST	/api/curriculums/{id}/pause/	학습 일시정지
POST	/api/curriculums/{id}/resume/	학습 재개
POST	/api/curriculums/{id}/complete/	커리큘럼 완료
POST	/api/curriculums/{id}/steps/{step_id}/complete/	특정 Step 완료

12.4 학습 대시보드

Method	Endpoint	설명
GET	/api/learning/dashboard/	요약, 현재 학습, 최근 활동
GET	/api/learning/curriculums/progress/	커리큘럼별 진행률
GET	/api/learning/current/	현재 학습 항목 상세
GET	/api/learning/roadmap/?curriculum_id={id}	선택 커리큘럼 로드맵

12.5 현재 구현되지 않은 초기 API

`/api/auth/logout/` , `/api/auth/me/` , `/api/interviews/...` , `/api/curriculum/generations/init/` , `/api/curriculum/generations/{id}/status/` , `/api/curriculums/{id}/retrygeneration/` 은 현재 URLConf 기준 구현되어 있지 않다.

13. 프론트엔드 설계

13.1 주요 페이지와 책임

페이지	책임
HomePage	목표 입력, 서비스 진입
ChatInterview	질문 선택지와 답변 수집
InterviewSummary	답변 요약, streaming 생성 요청, progress UI
CurriculumResult	AI preview 표시 및 저장
CurriculumDetailPage	저장된 커리큘럼 상세, 학습 시작/정지/완료
LearningDashboard	현재 학습, 커리큘럼별 진행률, 일정/로드맵 요약
UserSidebar	공통 사이드바, 커리큘럼 목록과 진행률

13.2 Streaming UI

`InterviewSummary.vue` 는 생성 중 `summary-generating-screen` 을 표시하고, `generationProgress` 를 progress bar의 width로 연결한다. 실제 서버 이벤트가 45% 이후 오래 머무르는 구간에서는 프론트엔드 interpolation으로 최대 82%까지 부드럽게 움직여 멈춘 느낌을 줄인다.

14. 비기능 요구사항·오류 처리

영역	요구사항	대응
성능	장시간 AI 생성 체감 대기 최소화	Streaming 상태, progress interpolation
신뢰성	부분 저장 방지	<code>transaction.atomic</code> , service 계층 분리
보안	사용자 데이터 소유권 분리	JWT 인증, user-scoped queryset
무결성	AI 결과가 실제 DB 근거와 연결되어야 함	identifier validation, FK 조회
사용성	로딩/빈 상태/실패가 명확해야 함	생성 메시지, error event, dashboard fallback
확장성	새 데이터원과 새 Topic 확장 가능	Topic Catalog, Course/ResourceTopic 연결

15. 테스트 및 검증 계획

계층	검증 항목	관련 파일
Unit/Service	AI 응답 저장, 상태 전환, 일정 생성, 대시보드 집계	apps/curriculum/tests.py , test_curriculum_learning_api.py
API	인증, 생성, streaming event, 학습 제어, 대시보드	apps/curriculum/test_learning_dashboard_api.py
Frontend	라우팅, 생성 대기 화면, 결과/상세/대시보드 렌더링	frontend/src/pages/user/
Scenario	데모 유저로 여러 커리큘럼 학습 상태 시연	scripts/seed_demo_user_data.py

15.1 발표용 데모 데이터

시연 안정성을 위해 발표용 유저와 여러 상태의 커리큘럼 데이터를 생성하는 seed 스크립트를 추가했다.

```
python scripts/seed_demo_user_data.py
```

- 진행 중 커리큘럼: 데이터 분석, 포트폴리오 발표 준비
- 완료 커리큘럼: Django REST API 백엔드 포트폴리오
- 일시정지 커리큘럼: 코딩테스트 알고리즘 루틴
- 초안 커리큘럼: SQL 리포팅 역량 강화

16. 협업·Git·WBS

16.1 그라운드룰

- 데일리 스크럼에서 어제 한 일, 오늘 할 일, 막힌 문제를 공유한다.
- 중요한 결정은 구두 합의로 끝나지 않고 문서화한다.
- 30분 이상 막히면 오류, 발생 상황, 추정 원인, 시도 내용을 공유한다.
- 기능 담당자를 정하되 핵심 로직은 팀 전체가 이해할 수 있게 공유한다.

16.2 Git 전략

항목	규칙
브랜치	main -> develop -> feature/fix/docs/refactor
직접 Push	main , develop 직접 push 지양
커밋	feat , fix , docs , refactor , chore
리뷰	핵심 기능은 PR 또는 팀 공유 후 병합

17. 설계 변경 이력

변경 항목	초기 방향	최종 방향	이유
일정 생성	시작 시 전체 Step 일정 생성	현재 시작한 Step만 일정 생성	중지/지연 시 미래 일정 무효화 방지
진행 상태	Schedule/Progress에서 계산	CurriculumStepProgress 별도 관리	계획, 상태, 일정, 기록 책임 분리
Topic	원본 키워드 직접 사용	표준 Topic + Alias + 검토 pipeline	중복 표현 통합과 검색 품질 관리
AI 추천	순수 LLM 중심	Topic Resolver + RAG + Agent	할루시네이션과 근거 부족 개선
생성 응답	일반 응답 완료까지 대기	StreamingHttpResponse	장시간 대기 UX 개선
저장	생성 즉시 저장	Preview 후 save-generated	사용자 확인과 데이터 무결성 확보
대시보드	후속 기능	학습 대시보드 API/화면 구현	저장된 커리큘럼을 실제 학습 관리로 연결

18. 한계와 향후 고도화

18.1 현재 한계

- 교육 데이터는 컴퓨터공학/AI 분야 중심이므로 다른 도메인 품질은 추가 검증이 필요하다.
- 추천 품질의 정량 평가 데이터셋과 사용자 실험 데이터가 제한적이다.
- 차시/영상 구간 단위까지 세밀하게 연결되지는 않았다.
- 백엔드 면담 세션 저장, 진단 퀴즈, topic별 사용자 숙련도 모델은 아직 구현되지 않았다.
- 비동기 작업 큐와 운영 배포 구조는 확정 구현 범위가 아니다.

18.2 고도화 우선순위

우선순위	고도화	기대 효과
1	추천 평가 데이터셋 구축	검색, 순서, 난이도 품질 정량 비교
2	비동기 Job Queue 및 generation 상태 조회	연결 단절/장시간 처리 대응 강화
3	면담 세션 DB 저장	질문/답변 이력 관리와 재개 기능
4	진단 퀴즈와 user_skill_profiles	사용자 수준 추정 정교화
5	학습 기록 기반 자동 재조정	실제 속도와 난이도에 맞춘 개인화
6	관리자 Topic/추천 품질 워크플로	후보 검수와 오류 수정 효율화
7	배포 인프라 확정	운영 환경, 로그, 백업, CI/CD, HTTPS 구성

부록 A. 실제 코드 기준 주요 경로

영역	경로
URL	config/urls.py , apps/accounts/urls.py
모델	apps/accounts/models.py , apps/curriculum/models.py
AI/RAG	ai/agent/ , ai/core/ , ai/search/ , ai/indexing/
커리큘럼 저장	apps/curriculum/services/curriculum_save_service.py
학습 진행	apps/curriculum/services/curriculum_learning_service.py
대시보드	apps/curriculum/services/learning_dashboard_service.py , apps/curriculum/learning_serializers.py
프론트 라우팅	frontend/src/router/index.js
스트리밍 프론트	frontend/src/api/curriculumApi.js , frontend/src/pages/user/InterviewSummary.vue